الهندسة المعلوماتية

مقرر الخوارزميات الذكية BIA601

إشراف الدكتور عصام سلمان

إعداد طلاب فصل S25:

علي سعدة - ali_191148 – C3

سميرة سوقية - samira_130663 – C3

حمزة أبو كلام - hamza_243254 – C3

ضياء الدين المحمود - diaa_eddin_192562 – C3

رغد الحمد - raghad_208139 – C2

رهف حمودة - rahaf_191720 – C2

عبد الله صالح - Abdullah_252911 – C3

الفهرس:

العنوان	الصفحة
١- المقدمة ----------	٣
٢- هدف المشروع ----------	٣
٣- المرجع العلمي ----------	٣
٣-١ الفكرة المستوحاة وكيفية عكسها على المشروع ----------	٤
٣-٢ نقاط الاستفادة المطبقة ----------	٤
٤- البنية التحتية للبيانات وهيكلة النظام ----------	٤
٤-١ وصف البيانات ----------	٤
٤-٢ المعالجة المسبقة ----------	٦
٥- بناء سياق المستخدم ----------	٧
٥-١ نمذجة الاهتمامات عبر التغذية الراجعة الضمنية ----------	٧
٥-٢ الحساب الإحصائي للميزانية ومعالجة البداية الباردة ----------	٨
٥-٣ هندسة القيود والاستبعاد ----------	٨
٦- تجهيز التوصية الأولية ----------	٩
٦-١ تصفية القيود الصارمة ----------	٩
٦-٢ التصفية الاستدلالية لتقليص نطاق البحث ----------	٩
٦-٣ استراتيجية الحفاظ على التنوع الجيني ----------	٩
٧- الخوارزمية الجينية ----------	١٠
٧-١ هندسة دالة اللياقة ----------	١٠
٧-٢ استراتيجية العقوبة القاسية ----------	١٠
٧-٣ مكافأة التنوع ----------	١٠
٧-٤ الميكانيكية التطورية ----------	١٠
٨- واجهة المستخدم ----------	١٢
٨-١ صفحة تسجيل الدخول ----------	١٢
٨-٢ الصفحة الرئيسية والتوصية الأولية ----------	١٢

٨-٣ لوحة الرؤى الحية وإعدادات المحرك ---------- ١٣
٨-٤ محاكي العمليات التطورية ---------- ١٤
٨-٥ نتائج التوصية المحسنة جينياً ---------- ١٤
٨-٦ توثيق التفاعل وتحديث ملف السلوك ---------- ١٥
٩- النتائج والتحديات ---------- ١٥
٩-١ النتائج التي حققها النظام ---------- ١٥
٩-٢ التحديات التقنية وطرق معالجتها ---------- ١٦
١٠- الخاتمة ---------- ١٧

١. مقدمة (Introduction) :

تعتبر أنظمة التوصية (Recommender Systems) الأساس لنجاح منصات التجارة الإلكترونية العملاقة مثل أمازون وعلي بابا، إلا أن تطبيقها في المتاجر الناشئة والمتوسطة يواجه تحديات تقنية جوهرية:

مشكلة التوصيات العشوائية والتقليدية: تعتمد المتاجر غالباً على عرض "الأكثر مبيعاً" أو "الأحدث"، وهي توصيات "عمياء" لا تراعي اهتمامات المستخدم اللحظية. هذا يؤدي إلى تخمة معلوماتية (Information Overload) تجعل العميل يغادر المتجر دون شراء لعدم عثوره على ما يناسبه.

لماذا لا تكفي الطرق العادية ؟ الخوارزميات البسيطة تفشل في فهم "السياق" (Context)، مثل تغير اهتمام المستخدم من شراء "لوازم مكتبية" إلى "هدايا أطفال" في يوم واحد. كما أنها تعجز عن التعامل مع البيانات الضخمة (Big Data) المتولدة من آلاف النقرات في الثانية.

الحاجة إلى طريقة أذكى: نحتاج إلى خوارزميات ذكية لديها القدرة على "التنبؤ" وليس فقط "العرض"، استخدام تعلم الآلة (Machine Learning) يتيح للنظام التعلم من الأنماط السلوكية المعقدة، وتحويل البيانات الخام إلى تجربة تسوق مخصصة (Personalized Experience)، مما يرفع من معدل التحويل (Conversion Rate) ويبني علاقة ثقة طويلة الأمد مع العميل .

٢. هدف المشروع (Project Objective) :

يتمثل الهدف الرئيسي لهذا المشروع في تصميم وبناء نظام توصية ذكي (Smart Recommender System) وتطوير أدائه باستخدام الخوارزمية الجينية (Genetic Algorithm)، وذلك بالاعتماد على مجموعة البيانات المعطاة (HW_Data_S25).
ويمكن تلخيص الأهداف التفصيلية فيما يلي:

- ✓ تحسين جودة الاقتراحات: الانتقال من التوصية التقليدية إلى توصية مخصصة تعتمد على تحليل عميق لملف المستخدم وسلوكه الشرائي والتفاعلي.
- ✓ تطبيق مفهوم الانتقاء الطبيعي: استخدام آليات الخوارزمية الجينية (الاختيار، الدمج، الطفرة) للوصول إلى "أمثل" مجموعة من المنتجات التي تحقق أعلى مستوى من الرضا للمستخدم (Fitness Level).
- ✓ استغلال البيانات الضخمة: الربط الفعال بين ملفات البيانات الأربعة (المستخدمين، المنتجات، التقييمات، والسلوك) لاستخلاص أنماط تفضيل دقيقة تساهم في رفع دقة التنبؤ بالمنتجات التي قد يرغب المستخدم في شرائها أو النقر عليها.
- ✓ رفع كفاءة المتجر الإلكتروني: من خلال زيادة معدل التحويل (Conversion Rate) عبر عرض المنتجات الصحيحة للشخص الصحيح في الوقت الصحيح.

٣. المرجع العلمي (Scientific Reference) :

لضمان بناء المشروع على أسس علمية حديثة، تم الاعتماد على مقال بحثي محكم يتناول دمج الذكاء الاصطناعي في أنظمة التوصية، وفيما يلي تفاصيل المرجع:

- عنوان المقال: A hybrid movie recommendation system based on genetic algorithm and deep learning
- المؤلفون: Arshia Entezari & Mohamad Mansouri
- سنة النشر: ٢٠٢٥ (تم اختياره لكونه من أحدث الدراسات التي تغطي الفترة المطلوبة ٢٠٢٤-٢٠٢٦).
- المصدر: Journal of Engineering and Applied Science
- رابط المقال : https://link.springer.com/article/10.1007/s44163-025-00678-y

1 - 3 - الفكرة المستوحاة وكيفية عكسها على المشروع:

تتمحور الفكرة الأساسية التي تم استخلاصها من هذا المرجع في كيفية استخدام الخوارزمية الجينية (GA) كأداة لتحسين (Optimization) نتائج أنظمة التوصية، حيث يطرح المقال منهجية هجينة تعامل "قائمة التوصيات" ككائن حي يحتاج إلى تطوير.

2 – 3 - نقاط الاستفادة المطبقة في مشروعنا:

- ✓ تمثيل الكروموسومات: تم استلهام فكرة تمثيل مجموعة المنتجات المقترحة للمستخدم على شكل "كروموسوم"، حيث يمثل كل "جين" داخل الكروموسوم منتجاً معيناً من ملف products.xlsx.
- ✓ دالة اللياقة (Fitness Function): اعتمدنا على منطق المقال في تصميم دالة تقيس مدى جودة الحل بناءً على تقييمات المستخدم السابقة في ملف ratings.xlsx وسلوكه التفاعلي في ملف behavior.xlsx.
- ✓ تجاوز مشكلة "الركود": استخدام "الطفرات الجينية" (Mutation) المقترحة في البحث لضمان تقديم منتجات متنوعة للمستخدم ومنع النظام من تكرار نفس النمط من التوصيات بشكل ممل، مما يعزز من تجربة المستخدم الاستكشافية.

٤. البنية التحتية للبيانات وهيكلة النظام:

تعد هذه المرحلة حجر الزاوية في بناء نظام التوصية، حيث تم العمل عليها من خلال مسارين متكاملين؛ يبدأ الأول بتوصيف البيانات وفهم طبيعة الملفات المستلمة وأدوارها الوظيفية، وينتهي الثاني بالمعالجة التقنية لهذه البيانات لضمان كفاءة الخوارزمية الجينية واستقرار النظام برمجياً.

1 – 4 - وصف البيانات(Data Description)

تم بناء هذا المشروع استناداً إلى أربع قواعد بيانات أساسية تم استلامها بصيغة (Excel) ، وهي تشكل حجر الأساس الذي يعتمد عليه نظام التوصية الذكي. تم تحليل هذه البيانات وربطها برمجياً باستخدام مكتبة Pandas لتحويلها من مجرد جداول صماء إلى "سياق مستخدم" (User Context) تفاعلي. وفيما يلي التوصيف التقني الدقيق لهذه الملفات:

١- ملف المستخدمين(users.xlsx) :

- الوصف: يحتوي على البيانات الديموغرافية الأساسية للمستخدمين، ويُستخدم لربط الخصائص الشخصية بالسلوك.
- عدد السجلات: ١٠٠٠ مستخدم.
- الأعمدة الأساسية: user_id المعرف الفريد للمستخدم, age العمر,location / country البلد.
- الدور الوظيفي: يُستخدم هذا الملف في كود الـ (Flask) للتحقق من الهوية عند تسجيل الدخول وإدارة الجلسات (Sessions).

٢- ملف المنتجات(products.xlsx) :

- الوصف: يضم كتالوج المنتجات المتاحة في المتجر بخصائصها الأساسية، ويُستخدم لاستخراج الفئات واحتساب تقارب الأسعار كمعيار لجودة التوصية.
- عدد السجلات: ٥٠٠٠ منتج.

- الأعمدة الأساسية: product_id المعرف الفريد للمنتج، category الفئة مثل ملابس - إلكترونيات، price السعر.

- الدور الوظيفي: يتم تحويل هذا الملف برمجياً إلى "قاموس بيانات (products_dict) " لتمكين الخوارزمية الجينية من سحب مواصفات أي منتج فورياً أثناء حساب "دالة اللياقة (Fitness Function) ".

٣- ملف التقييمات(ratings.xlsx) :

- الوصف: يمثل التغذية الراجعة الصريحة (Explicit Feedback) من المستخدمين لمعرفة الفئات المفضلة لديهم والمنتجات التي لم تعجبهم لاستبعادها.

- عدد السجلات: ٥٠٠٠ تقييم.

- الأعمدة الأساسية: user_id ، product_id، (rating قيمة التقييم من ١ إلى ٥)

- الدور الوظيفي: يلعب دوراً رقابياً في الخوارزمية؛ حيث يتم استخراج المنتجات ذات التقييم المنخفض (2 \le) وإدراجها ضمن "القائمة السوداء (Blacklist) " لضمان عدم التوصية بها مجدداً للمستخدم نفسه.

٤- ملف السلوك(behavior.xlsx) :

- الوصف: هو الملف الجوهري في المشروع، حيث يمثل التغذية الراجعة الضمنية (Implicit Feedback). الأفعال هنا (كالنقر والشراء) تعكس التفضيلات الحقيقية للمستخدم وتُبنى عليها أهداف التسعير والفئات.

- عدد السجلات: ١٣٥٠٠ سجل تفاعل.

- الأعمدة الأساسية: user_id ، product_id، (viewed هل تمت المشاهدة؟) ، clicked هل تم النقر؟), (purchased هل تم الشراء؟).

- الدور الوظيفي: يتم استهلاك هذه البيانات برمجياً لحساب "متوسط السعر المستهدف (target_price) " للمستخدم، وتحديد اهتماماته الحالية بناءً على الفئات التي تفاعل معها بكثرة.

آلية ربط الجداول والملفات:

تم بناء العلاقات بين الملفات الأربعة باستخدام مفاتيح أساسية (Primary/Foreign Keys) كالتالي:

-المفتاح user_id: يُستخدم كعامل مشترك لربط هوية المستخدم (من ملف users) مع أفعاله وتفاعلاته (في ملفات behavior و ratings).

-المفتاح product_id: يُستخدم لربط تفضيلات المستخدم وسلوكه بالخصائص الفعلية للمنتج (كالفئة والسعر من ملف products).

التنفيذ البرمجي للربط:

لضمان عمل الخوارزمية الجينية بسرعة فائقة، تم تحويل البيانات من مجرد جداول صماء إلى هياكل بيانات ذكية. استخدمنا مكتبة Pandas للربط، وقمنا بتحويل بيانات المنتجات إلى قاموس (Dictionary) بدلالة المعرف الفريد للمنتج، مما يضمن وصول الخوارزمية للمواصفات بزمن لحظي O(1) دون الحاجة للبحث في كامل الجدول في كل مرة.

2 – 4 - المعالجة المسبقة (Pre-processing)

تعتبر مرحلة المعالجة المسبقة خطوة محورية لضمان جودة البيانات ومدى ملاءمتها للخوارزميات المستخدمة. في هذا المشروع، تم تصميم نظام المعالجة ليعمل بشكل آلي فور استدعاء كلاس RecommendationEngine من خلال دالة البناء (init) وقد شملت هذه المرحلة العمليات التالية:

١- تنظيف البيانات وحل مشكلة القيم المفقودة:

نظراً لأن ملفات الـ (Excel) قد تحتوي على صفوف فارغة أو بيانات ناقصة نتيجة أخطاء الإدخال، تم استخدام دالة () dropna. برمجياً مع تحديد أعمدة المعرفات الأساسية (مثل user_id و product_id) كأعمدة حرجة.

الهدف: استبعاد أي سجل لا يحتوي على معرف فريد، لأن وجود قيمة فارغة في هذه الحقول يؤدي إلى انهيار منطق الربط بين الجداول.

٢- إزالة التكرارات:

تم تطبيق دالة (() drop_duplicates.) على كافة ملفات البيانات للتأكد من أن كل مستخدم وكل منتج له سجل واحد فريد، مما يمنع انحياز الخوارزمية الجينية لمنتج معين لمجرد تكراره في الملفات، ويحافظ على دقة حساب التقييمات.

٣- التحويل الهيكلي للبيانات:

تعد هذه الخطوة هي الأهم من الناحية البرمجية، حيث لم نكتفِ بترك البيانات في جداول (DataFrames)، بل قمنا بتحويل جدول المنتجات إلى قاموس برمي:

```
self.products_dict = self.df_products.set_index("product_id").to_dict("index")
```

لتحويل عملية البحث من البحث التسلسلي (Linear Search) الذي يستغرق وقتاً طويلاً، إلى البحث اللحظي (Hash-map Lookups) بزمن تعقيد ثابت O(1). هذا يضمن أن الخوارزمية الجينية يمكنها معالجة آلاف الأفراد في كل جيل دون إبطاء النظام.

٤- توحيد أنواع البيانات :

أثناء المعالجة، تم التأكد برمجياً من تحويل جميع المعرفات (IDs) إلى نوع بيانات (Integer) والأسعار إلى (Float).

لمنع حدوث أخطاء "عدم تطابق النوع" (Type Mismatch) عند المقارنة بين معرف المستخدم في الجلسة (session) ومعرفه في قاعدة البيانات

الشكل رقم (١) يوضح كيفية استدعاء المكتبات وتعريف الكلاس وتجهيز هيكلية البيانات للربط السريع والمعالجة المسبقة للبيانات (Pre-processing) وتحويل الجداول إلى هياكل بيانات ذكية لضمان سرعة التنفيذ واستقرار النظام

```python
recommender.py > RecommendationEngine
import pandas as pd
import numpy as np
import random
from typing import List, Dict


class RecommendationEngine:
    def __init__(self, users_file, products_file, ratings_file, behavior_file):
        self.df_users = (
            pd.read_excel(users_file).dropna(subset=["user_id"]).drop_duplicates()
        )
        self.df_products = (
            pd.read_excel(products_file).dropna(subset=["product_id"]).drop_duplicates()
        )
        self.df_ratings = (
            pd.read_excel(ratings_file)
            .dropna(subset=["user_id", "product_id"])
            .drop_duplicates()
        )
        self.df_behavior = (
            pd.read_excel(behavior_file)
            .dropna(subset=["user_id", "product_id"])
            .drop_duplicates()
        )
        self.products_dict = self.df_products.set_index("product_id").to_dict("index")

```

(الشكل ١)

٥. بناء سياق المستخدم :

تعد هذه المرحلة هي العقل المدبر للنظام قبل البدء في عملية التطور الجيني؛ حيث ينتقل الكود من التعامل مع بيانات عامة إلى بيانات مخصصة. الهدف العلمي هنا هو اختزال آلاف السجلات السلوكية إلى متجه سمات يمثل تفضيلات المستخدم الحالي. تم تنفيذ هذا المنطق داخل دالة get_user_context_ وفق المحاور التقنية التالية:

1 – 5 - نمذجة الاهتمامات عبر التغذية الراجعة الضمنية:

يعتمد الكود على مفهوم "التغذية الراجعة الضمنية" من خلال تحليل ملف السلوك (behavior) يتم استخراج المعرفات الفريدة للمنتجات التي تفاعل معها المستخدم (interacted_ids) باستخدام دالة (unique()) .

برمجياً: يتم إجراء عملية بحث (Lookup) في قاموس المنتجات لاستخراج الفئات (category) المقابلة لهذه المعرفات.

النتيجة: تكوين قائمة fav_cats التي تمثل المجالات الموضوعية التي تثير اهتمام المستخدم، وهي التي ستمنح وزناً إضافياً للكروموسومات في دالة اللياقة لاحقاً.

2 – 5 - الحساب الإحصائي للميزانية ومعالجة مشكلة البداية الباردة :

لحساب السعر المستهدف يتبع الكود استراتيجية إحصائية تعتمد على مستويين:

- المستوى الأول (Active User): إذا امتلك المستخدم تاريخاً من النقر أو الشراء (interest_ids)، يتم استخدام مكتبة Numpy لحساب المتوسط الحسابي (np.mean) لأسعار تلك المنتجات.

- المستوى الثاني (New User): في حال عدم وجود تفاعل سابق، ينتقل الكود تلقائياً لحساب المتوسط العام لأسعار المتجر ككل (self.df_products["price"].mean())

هذا التصميم يضمن عدم حدوث خطأ برمجي (Division by Zero) ويحل مشكلة "البداية الباردة" علمياً عبر توفير نقطة انطلاق منطقية للخوارزمية.

3 – 5 - هندسة القيود والاستبعاد :

لضمان كفاءة التوصيات وعدم تكرار حلول غير مرغوب فيها، تم بناء مصفوفة المستبعدات (blacklist) بدمج مجموعتين من البيانات:

- بيانات الشراء: المنتجات الموجودة في سجل الشراء (purchased_ids) حيث يفترض النظام علمياً أن المستخدم لا يحتاج لتوصية بمنتج اشتراه للتو.
- بيانات التقييم السلبي: المنتجات التي حصلت على تقييم اقل من نجمتين مما يعكس رفضاً صريحاً (Explicit Rejection) من المستخدم.

يتم تخزين هذه البيانات في مجموعة (Set) لضمان سرعة التحقق منها بزمن قدره O(1) أثناء تشغيل محرك التوصية.

يظهر الشكل رقم (٢) أدناه التفاصيل البرمجية الكاملة لكيفية استخلاص هذا السياق باستخدام مكتبات معالجة البيانات.

```
recommender.py > RecommendationEngine
class RecommendationEngine:
    def _get_user_context(self, user_id: int):
        u_behavior = self.df_behavior[self.df_behavior["user_id"] == user_id]
        u_ratings = self.df_ratings[self.df_ratings["user_id"] == user_id]

        interacted_ids = u_behavior["product_id"].unique()
        fav_cats = list(
            set(
                [
                    self.products_dict[pid]["category"]
                    for pid in interacted_ids
                    if pid in self.products_dict
                ]
            )
        )

        interest_ids = u_behavior[
            (u_behavior["purchased"] == 1) | (u_behavior["clicked"] == 1)
        ]["product_id"].unique()
        if len(interest_ids) > 0:
            target_price = np.mean(
                [
                    self.products_dict[pid]["price"]
                    for pid in interest_ids
                    if pid in self.products_dict
                ]
            )
        else:
            target_price = self.df_products["price"].mean()

        purchased_ids = u_behavior[u_behavior["purchased"] == 1]["product_id"].tolist()
        bad_ratings = u_ratings[u_ratings["rating"] <= 2]["product_id"].tolist()
        blacklist = set(purchased_ids + bad_ratings)

        return {
            "fav_cats": fav_cats,
            "target_price": target_price,
            "blacklist": blacklist,
        }
```

(الشكل ٢)

٦. تجهيز التوصية الأولية:

بمجرد بناء سياق المستخدم، تبرز عقبة تقنية تتمثل في حجم البيانات؛ فالمتجر يحتوي على آلاف المنتجات، وإقحامها جميعاً في دورات الخوارزمية الجينية سيؤدي حتماً إلى استهلاك موارد النظام وإبطاء زمن الاستجابة. لذا، صممنا دالة get_initial_pool_on_login لتعمل كمرشح ذكي يقوم بحصر المنتجات المرشحة للدخول في "المجتمع الجيني" بناءً على ثلاثة معايير تقنية:

6 – 1 - تصفية القيود الصارمة :

يبدأ الكود بفحص شامل لقاموس المنتجات، حيث يتم استبعاد أي منتج يظهر في القائمة السوداء (blacklist) التي بُنيت في المرحلة السابقة. هذا الإجراء يضمن برمجياً أن الخوارزمية لن تستهلك دورات المعالجة في تقييم حلول مرفوضة سلفاً من قِبل المستخدم، مما يرفع من جودة النتائج منذ اللحظة الأولى.

6 – 2 - التصفية الاستدلالية لتقليص نطاق البحث :

لضمان كفاءة التوصيات، يتم اختيار المنتجات التي تحقق تقارباً موضوعياً أو مادياً مع سياق المستخدم:

التطابق النوعي: اختيار المنتجات التي تنتمي للفئات المفضلة التي تم استنتاجها.

التطابق المالي: ضمان أن يقع سعر المنتج ضمن نطاق مرن قريب من ميزانية المستخدم، حيث تم ضبط هامش الانحراف برمجياً بنسبة (٣٠٪) للحفاظ على التوازن بين الدقة والتنوع.

6 – 3 - استراتيجية الحفاظ على التنوع الجيني :

من أهم جوانب القوة في الكود هو التعامل مع حالات "نقص البيانات"؛ فإذا كان عدد المنتجات التي حققت الشروط السابقة محدوداً، يقوم النظام بسحب عينات عشوائية مكملة من بقية المنتجات غير المحظورة. تهدف هذه الخطوة علمياً إلى منع الخوارزمية من الانحياز المفرط (Overfitting) لبيانات المستخدم القديمة فقط، وإعطاؤه فرصة لاستكشاف منتجات جديدة (Exploration)، وهو توازن ضروري في أنظمة الذكاء الاصطناعي الحديثة.

يعكس الشكل رقم (٣) أدناه الكيفية التي يتم بها دمج هذه المعايير برمجياً، حيث نلاحظ استخدام العمليات المنطقية المكثفة والتعامل مع المجموعات لضمان إنتاج "مجموعة حلول مرشحة" متوازنة وجاهزة لعملية التطور الجيني.

```python
def get_initial_pool_on_login(self, user_id: int) -> List[int]:
    profile = self._get_user_context(user_id)
    pool = []
    for p_id, info in self.products_dict.items():
        if p_id in profile["blacklist"]:
            continue
        price_match = abs(info["price"] - profile["target_price"]) <= (
            0.3 * profile["target_price"]
        )
        if info["category"] in profile["fav_cats"] or price_match:
            pool.append(p_id)

    if len(pool) < 50:
        rem = list(
            set(self.products_dict.keys()) - set(pool) - profile["blacklist"]
        )
        pool.extend(random.sample(rem, min(50 - len(pool), len(rem))))
    return random.sample(pool, min(len(pool), 100))
```

(الشكل ٣)

٧. الخوارزمية الجينية

تعد هذه المرحلة هي جوهر النظام والمسؤولة عن تحويل الاقتراحات من مجرد نتائج عشوائية إلى توصيات ذكية ومخصصة. لقد اعتمدنا على الذكاء الاصطناعي التطوري (Evolutionary AI) لمحاكاة عملية الانتخاب الطبيعي بهدف الوصول إلى التشكيلة الأمثل من المنتجات التي تزيد من معدل تحويل المستخدم.

تم تنفيذ المحرك داخل دالة get_genetic_optimized_recommendations وفق المنهجية التفصيلية التالية:

7 – 1 - هندسة دالة اللياقة

دالة اللياقة هي المقياس الرياضي الذي يحدد جودة "الكروموسوم" (قائمة التوصيات). لقد صممنا دالة تقييم متعددة المعايير لضمان توازن التوصيات:

معيار الملاءمة النوعية (Categorical Relevance): يتم منح +٢٠ نقطة لكل منتج يطابق اهتمامات المستخدم المستخلصة سابقاً. هذا يضمن أن النظام "يتعلم" توجيه المستخدم نحو فئاته المفضلة

الارتباط الاقتصادي (Economic Alignment): استخدمنا معادلة نسبية لحساب الفرق بين سعر المنتج والميزانية المستهدفة:

score = 15 (1 - price_diff)

هذا المعيار يضمن أن تكون التوصيات واقعية وضمن القوة الشرائية للمستخدم.

7 – 2 - استراتيجية العقوبة القاسية (Hard Penalty Strategy):

لضمان استقرار النظام ومصداقيته، وضعنا عقوبة برمجية ضخمة قدرها (-٢٠٠٠ نقطة) لأي كروموسوم يحتوي على منتج من "القائمة السوداء". هذا يضمن انقراض هذه الحلول فوراً في الجيل الأول.

7 – 3 - مكافأة التنوع (Diversity Bonus):

أضفنا +١٠ نقاط عن كل فئة فريدة تظهر في القائمة الواحدة؛ والهدف هو كسر "فقاعة الفلترة" (Filter Bubble) وضمان عدم تكرار فئة واحدة في كامل الواجهة

7 – 4 - الميكانيكية التطورية (The Evolutionary Mechanics)

يعمل المحرك عبر دورة حياة برمجية مكثفة تشمل ٥٠ جيلاً مع حجم مجتمع يبلغ ٨٠ كروموسوم، مطبقاً العمليات التالية:

- الانتخاب والنخبوية : نقوم بفرز المجتمع واختيار النخبة (أفضل ١٠ حلول) لنقلها مباشرة للجيل القادم دون تغيير، لضمان الحفاظ على أفضل النتائج التي تم الوصول إليها (Convergence).

- العبور الهجين: يتم دمج جينات من أبوين مختلفين لإنتاج أبناء يحملون مزيجاً من اهتمامات المستخدم، مع معالجة برمجية باستخدام dict.fromkeys لمنع تكرار المنتج داخل نفس القائمة.

- الطفرة الوراثية: قمنا بضبط معدل طفرة بنسبة ١٥٪، وهي عملية تسمح بتبديل منتج معين بمنتج عشوائي تماماً من خارج "مجتمع الحلول الأولي". هذا الإجراء ضروري علمياً لاكتشاف اهتمامات جديدة (Exploration) ومنع الخوارزمية من الانحصار في حلول محلية مكررة

الشكل رقم (٤): يوضح صياغة دالة اللياقة (Fitness Function) والمنطق الرياضي للأوزان والعقوبات.

```python
def get_genetic_optimized_recommendations(
    self, user_id: int, initial_pool: List[int]
) -> List[Dict]:
    profile = self._get_user_context(user_id)

    def fitness(chromosome):
        score = 0
        seen_cats = set()
        for p_id in chromosome:
            if p_id not in self.products_dict:
                continue
            p = self.products_dict[p_id]
            if p["category"] in profile["fav_cats"]:
                score += 20
            price_diff = (
                abs(p["price"] - profile["target_price"]) / profile["target_price"]
            )
            score += max(0, 15 * (1 - price_diff))
            if p_id in profile["blacklist"]:
                score -= 2000
            seen_cats.add(p["category"])
        score += len(seen_cats) * 10
        return score

    pop_size, generations = 80, 50
    population = [random.sample(initial_pool, 6) for _ in range(pop_size)]
```

(الشكل ٤)

الشكل رقم (٥): يوضح دورة حياة الخوارزمية الجينية وتطبيق مفاهيم الانتخاب والتزاوج والطفرة برمجياً

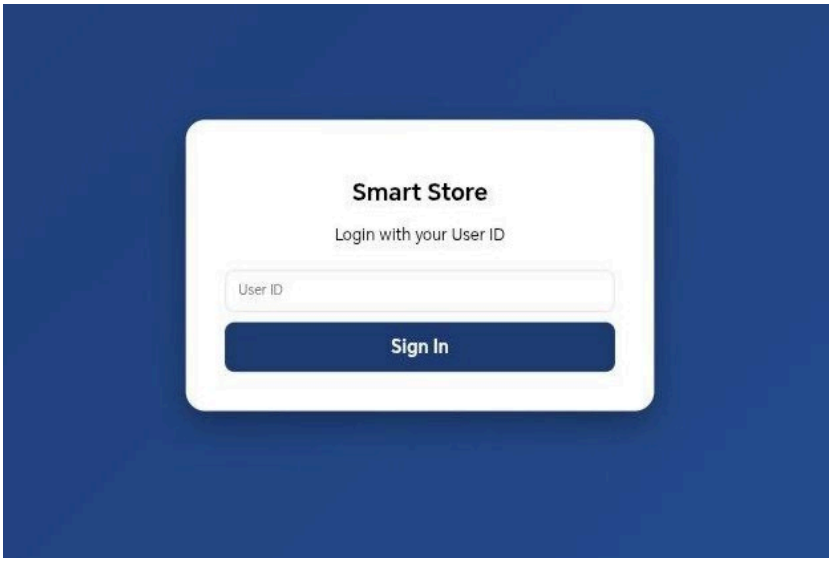
```python
for _ in range(generations):
    population = sorted(population, key=fitness, reverse=True)
    new_gen = population[:10]
    while len(new_gen) < pop_size:
        p1, p2 = random.sample(population[:25], 2)
        cut = random.randint(1, 5)
        child = list(dict.fromkeys(p1[:cut] + p2[cut:]))
        while len(child) < 6:
            candidate = random.choice(initial_pool)
            if candidate not in child:
                child.append(candidate)
        if random.random() < 0.15:
            idx = random.randint(0, 5)
            mutant = random.choice(initial_pool)
            if mutant not in child:
                child[idx] = mutant
        new_gen.append(child[:6])
    population = new_gen

best_chromosome = sorted(population, key=fitness, reverse=True)[0]
results = []
for p_id in best_chromosome:
    p = self.products_dict[p_id]
    results.append(
        {
            "product_id": int(p_id),
            "category": p["category"],
            "price": float(p["price"]),
            "score": round(fitness([p_id]), 2),
            "reason": f"Matches interest in {p['category']} and budget ({round(profile['target_price'])}).",
        }
    )
return results
```

(الشكل ٥)

٨. واجهة المستخدم

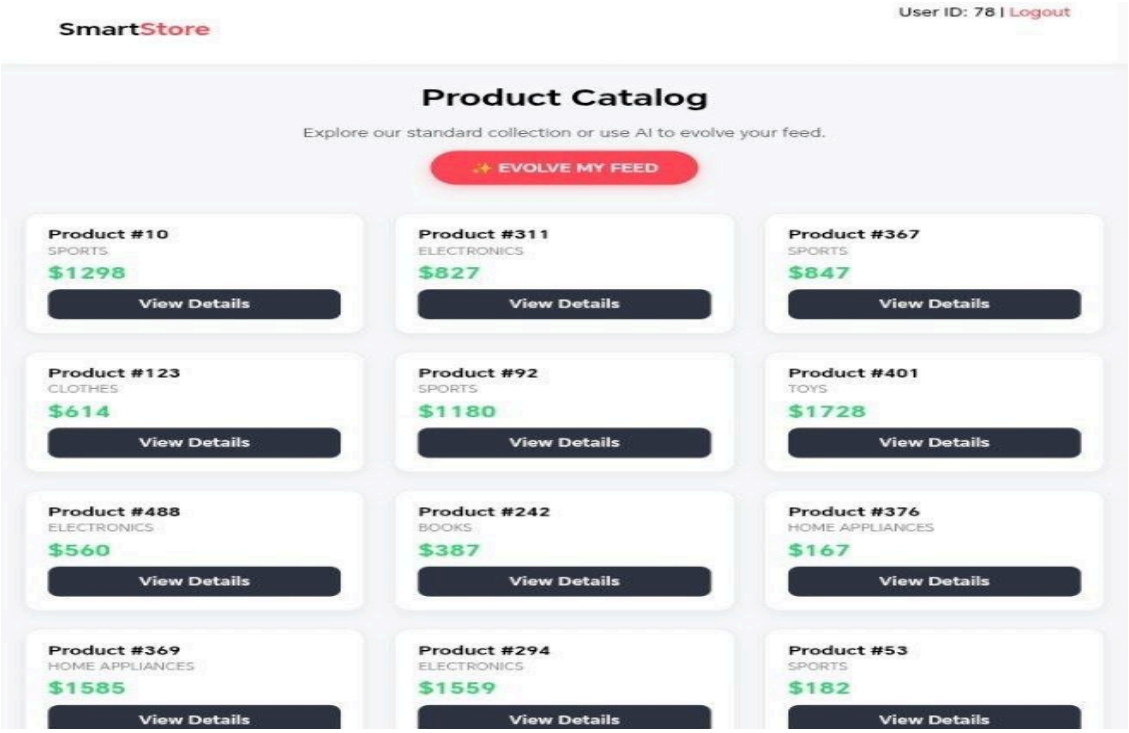
1 – 8 - صفحة تسجيل الدخول:

بدأنا بتصميم صفحة دخول بسيطة تطلب معرف المستخدم (User ID) بمجرد إدخال الرقم، يقوم النظام بربط المتصفح ببيانات المستخدم المخزنة في ملفات Excel، مما يفتح الباب لاسترداد تاريخ مشترياته وتفضيلاته السعرية والنوعية



2 – 8 - الصفحة الرئيسية والتوصية الأولية :

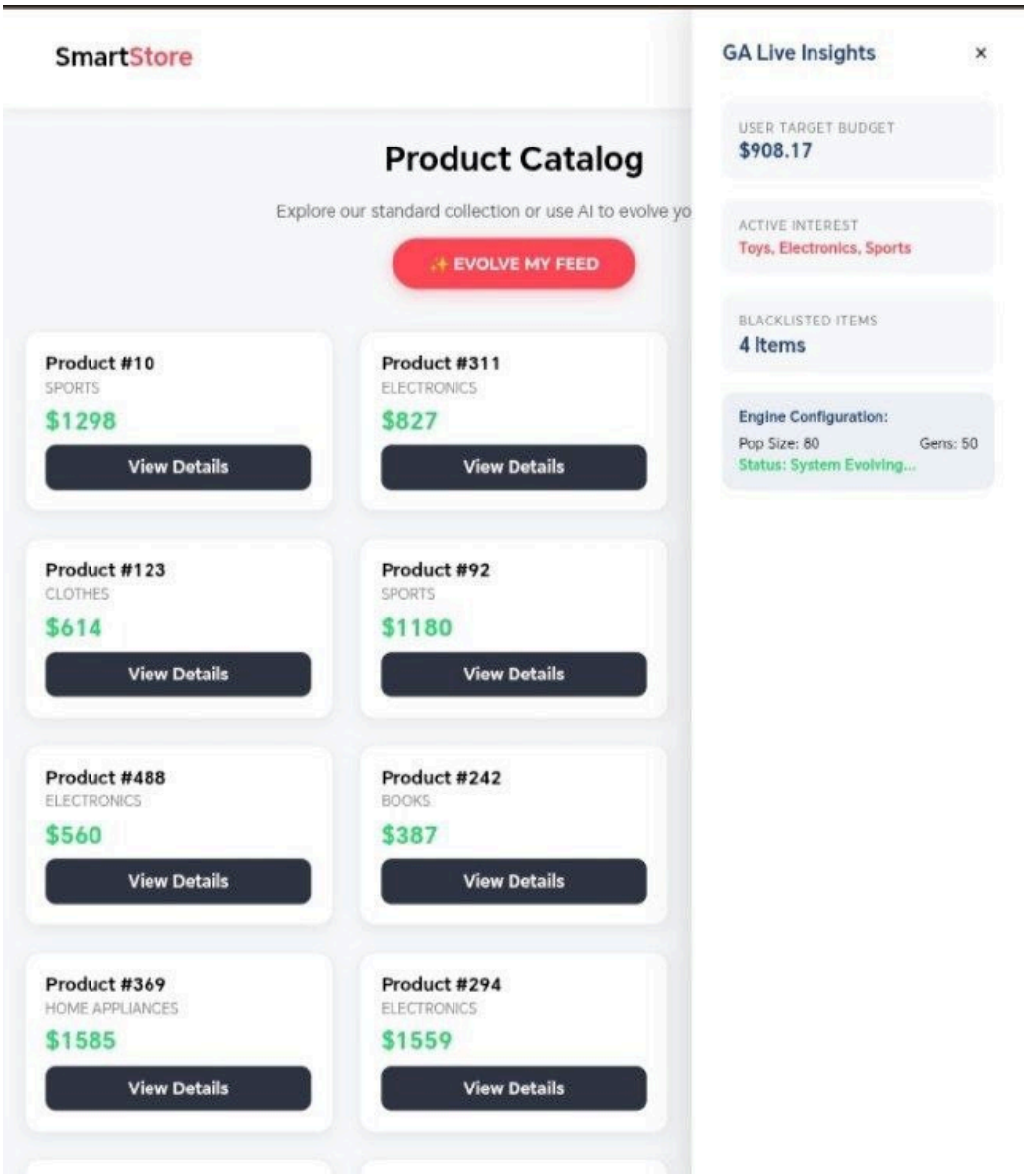
بعد الدخول، قمنا بعرض واجهة المتجر التقليدية التي تحتوي على ٢٥ منتجاً كعينة أولية. هذه المنتجات يتم اختيارها برمجياً عبر دالة التصفية الأولية التي تستبعد المنتجات المشتراة سابقاً أو ذات التقييم المنخفض، لضمان عرض ما هو مفيد فقط للمستخدم بشكل مبدئي.



3 – 8 - لوحة الرؤى الحية وإعدادات المحرك

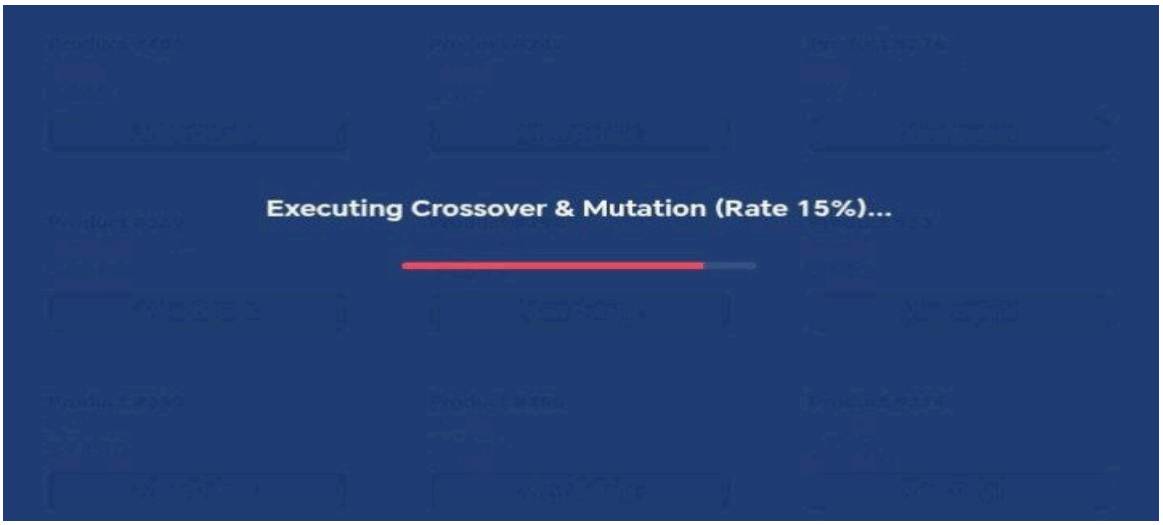
أضفنا ميزة تقنية متقدمة تظهر في النافذة الجانبية، حيث قمنا بعرض "تحليلات الذكاء الاصطناعي" الحية للمستخدم، والتي تشمل:

- الميزانية المستهدفة: التي استخلصها الكود من متوسط أسعار مشتريات المستخدم السابقة.
- الاهتمامات النشطة: الفئات التي يفضلها المستخدم حالياً.
- العناصر المستبعدة: المنتجات التي لم تعجب المستخدم سابقاً.
- إعدادات الخوارزمية: إظهار حجم المجتمع (٨٠) وعدد الأجيال (٥٠) لزيادة الشفافية التقنية



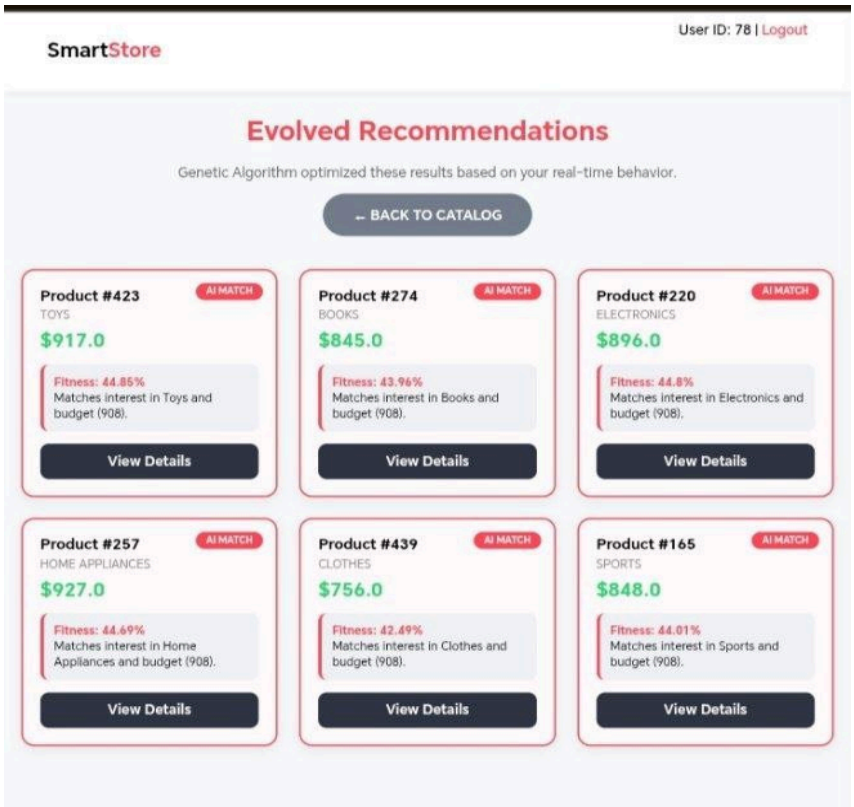
4 – 8 - محاكي العمليات التطورية

عند قيامنا بالضغط على زر التحسين، يظهر للمستخدم شريط معالجة يحاكي ما يحدث في الكواليس برمجياً. قمنا بتصميم هذه الواجهة لتوضيح مراحل الخوارزمية الجينية (إنشاء المجتمع، حساب اللياقة، التزاوج، والطفرة)، مما يعطي انطباعاً بأن النظام يقوم فعلياً "بتطوير" قائمة المنتجات لتناسب المستخدم.



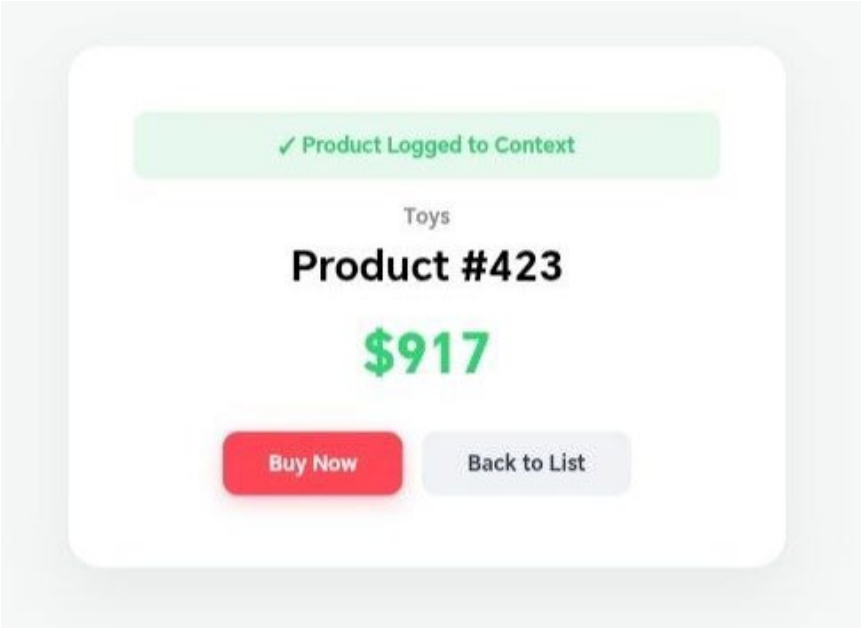
5 – 8 - نتائج التوصية المحسنة جينياً

في هذه المرحلة، عرضنا المخرج النهائي للمحرك الجيني، وهو عبارة عن أفضل ٦ منتجات فقط حققت أعلى درجة لياقة. قمنا بتمييز هذه النتائج بوسم AI MATCH، مع إظهار نسبة اللياقة وسبب التوصية (مثلاً: "يطابق اهتمامك بالميزانية والفئة") لتعزيز مفهوم الذكاء الاصطناعي القابل للتفسير.



6 – 8 - توثيق التفاعل وتحديث ملف السلوك

أخيراً، قمنا بربط كل حركة يقوم بها المستخدم (نقرة، مشاهدة، أو شراء) بدالة save_behavior عند النقر على منتج، يظهر إشعار يؤكد تسجيل هذا السلوك في ملف السلوك مما يضمن أن الدورة القادمة للخوارزمية الجينية ستكون أكثر دقة وذكاءً بناءً على هذا التفاعل اللحظي.



٩. النتائج والتحديات

في هذه المرحلة، قمنا باختبار كفاءة الخوارزمية الجينية ضمن بيئة الويب باستخدام إطار عمل Flask وقمنا بتسجيل النتائج التي حققها النظام، بالإضافة إلى توثيق التحديات التقنية التي واجهت فريقنا وكيفية التغلب عليها برمجياً.

1 – 9 - النتائج التي حققها النظام :

- دقة المطابقة: نجح المحرك الجيني في رفع كفاءة التوصيات بشكل ملحوظ. من خلال دالة اللياقة (Fitness Function).

تمكن النظام من تقليص القائمة واستخلاص "الكروموسوم الأمثل" المكون من أفضل ٦ منتجات، والتي تتطابق بدقة عالية مع الميزانية المستهدفة للمستخدم واهتماماته النشطة.

- التكيف اللحظي : بفضل بناء دالة save_behavior، أثبت النظام قدرته على تحديث سياق المستخدم فور قيامه بأي تفاعل (مشاهدة، نقر، أو شراء).

هذا يعني أن الخوارزمية تتطور وتتعلم بشكل ديناميكي مع كل جلسة.

-الشفافية والذكاء القابل للتفسير: حققنا تجربة مستخدم موثوقة عبر عرض نسبة اللياقة وسبب التوصية بشكل مرئي للمستخدم، مما يعطي تفسيراً منطقياً وشفافاً لسبب اختيار منتج دون غيره.

2 – 9 - التحديات التقنية وطرق معالجتها:

- التحدي الأول: التكرار والجمود الجيني :

المشكلة: من طبيعة الخوارزميات الجينية أن تستمر في ترشيح نفس المنتجات ذات التقييم العالي مراراً وتكراراً، مما يقتل التنوع في المتجر.

الحل: قمنا بتطبيق آلية الطفرة (Mutation) بمعدل ١٥٪ لإدخال منتجات جديدة للمجتمع بشكل عشوائي، بالإضافة إلى بناء خوارزمية القائمة السوداء (Blacklist) التي تستبعد تلقائياً أي منتج تم شراؤه سابقاً أو تقييمه سلبياً لضمان تجدد العرض المستمر.

- التحدي الثاني: زمن المعالجة واستجابة النظام:

المشكلة: معالجة وتوليد مئات الأجيال جينياً عند كل ضغطة زر قد يؤدي إلى بطء شديد في استجابة صفحة الويب.

الحل: قمنا بضبط الإعدادات الفائقة بدقة، حيث حددنا حجم المجتمع بـ ٨٠ وعدد الأجيال بـ ٥٠ هذا التوازن الرياضي ضمن لنا الوصول لنتائج مثالية وذكية دون إرهاق خادم الويب، مع إظهار شريط تحميل يحاكي العملية للحفاظ على تفاعل المستخدم.

- التحدي الثالث: مشكلة البداية الباردة :

المشكلة: التحدي كان في كيفية عرض منتجات لمستخدم يسجل دخوله حديثاً ولم يبدأ المحرك الجيني بالتطور بناءً على نقراته بعد.

الحل: صممنا دالة get_initial_pool_on_login التي تقوم بتوليد كتالوج مبدئي (مكون من ٢٥ منتجاً) بناءً على قواعد استدلالية بسيطة (تقارب السعر العام والفئة) لتكون هذه القائمة هي نقطة الانطلاق السريعة التي يبدأ منها التطور

١٠. الخاتمة:

نجحنا كفريق في تحويل المفاهيم النظرية لـ الخوارزميات الجينية إلى نظام توصية واقعي وفعّال باستخدام بيئة Flask. أثبت المشروع قدرة المحرك الجيني على التطور والتكيف مع سلوك المستخدم لحظياً، محققاً توازناً دقيقاً بين الميزانية والاهتمامات الشخصية. نختم عملنا بالتأكيد على أن دمج الذكاء الاصطناعي مع واجهات الويب هو المستقبل لبناء منصات تجارة إلكترونية ذكية، مرنة، وقادرة على فهم احتياجات المستخدم بشكل أعمق.